NAME: Prabhkirat Singh Marwah

SEC: B

BATCH: B4 ¶

ROLL NO: 47

ML LAB : Practical 4

In [ ]:

In [ ]:

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib as plt
```

In [2]:
```
df=pd.read_csv('winequality-red.xls')
```

In [3]:
```
df.head()
```

Out[3]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	Id
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	0
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	1
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	2
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	3
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	4

In [4]:
```
df.tail()
```

Out[4]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	Id
1138	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0	6	1592
1139	6.8	0.620	0.08	1.9	0.068	28.0	38.0	0.99651	3.42	0.82	9.5	6	1593
1140	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5	5	1594
1141	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2	6	1595
1142	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2	5	1597

In [5]:
```
df.shape
```

Out[5]:
```
(1143, 13)
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   fixed acidity         1143 non-null   float64
 1   volatile acidity      1143 non-null   float64
 2   citric acid           1143 non-null   float64
 3   residual sugar        1143 non-null   float64
 4   chlorides             1143 non-null   float64
 5   free sulfur dioxide   1143 non-null   float64
 6   total sulfur dioxide  1143 non-null   float64
 7   density               1143 non-null   float64
 8   pH                    1143 non-null   float64
 9   sulphates             1143 non-null   float64
 10  alcohol               1143 non-null   float64
 11  quality               1143 non-null   int64  
 12  Id                    1143 non-null   int64  
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

In [7]:
```
df.describe()
```

Out[7]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	
count	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	114
mean	8.311111	0.531339	0.268364	2.532152	0.086933	15.615486	45.914698	0.996730	3.311015	0.657708	10.442111	5.657043	8(
std	1.747595	0.179633	0.196686	1.355917	0.047267	10.250486	32.782130	0.001925	0.156664	0.170399	1.082196	0.805824	4(
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.740000	0.330000	8.400000	3.000000	
25%	7.100000	0.392500	0.090000	1.900000	0.070000	7.000000	21.000000	0.995570	3.205000	0.550000	9.500000	5.000000	4(
50%	7.900000	0.520000	0.250000	2.200000	0.079000	13.000000	37.000000	0.996680	3.310000	0.620000	10.200000	6.000000	79
75%	9.100000	0.640000	0.420000	2.600000	0.090000	21.000000	61.000000	0.997845	3.400000	0.730000	11.100000	6.000000	12(
max	15.900000	1.580000	1.000000	15.500000	0.611000	68.000000	289.000000	1.003690	4.010000	2.000000	14.900000	8.000000	15(

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
Id                      0
dtype: int64
```

In [9]:
```
df.duplicated().sum()
```

Out[9]:
```
np.int64(0)
```

In [10]:
```
df.columns
```

Out[10]:
```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol', 'quality', 'Id'],
      dtype='object')
```

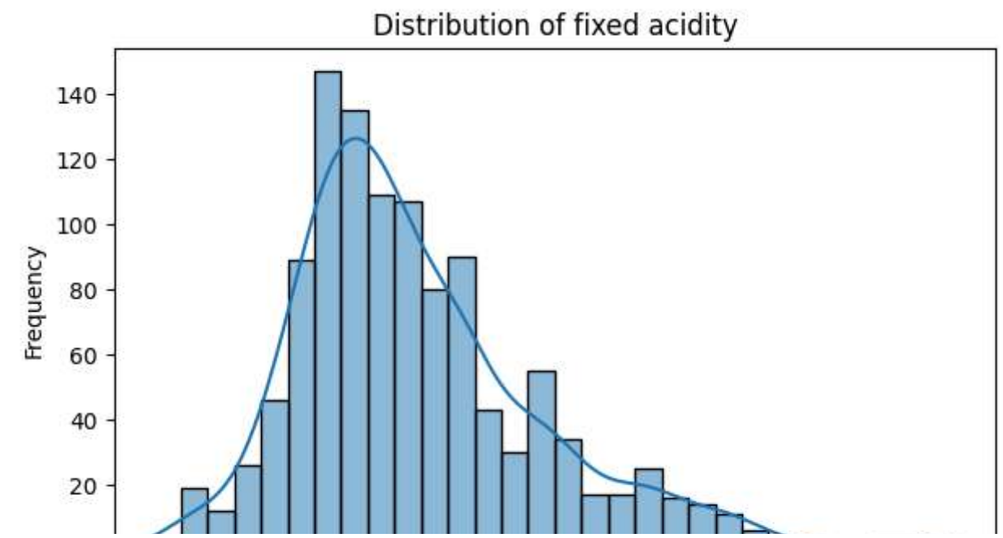
Univariate Ananlysis

In [11]:

```
#Histogram+KDE
import matplotlib.pyplot as plt
import seaborn as sns

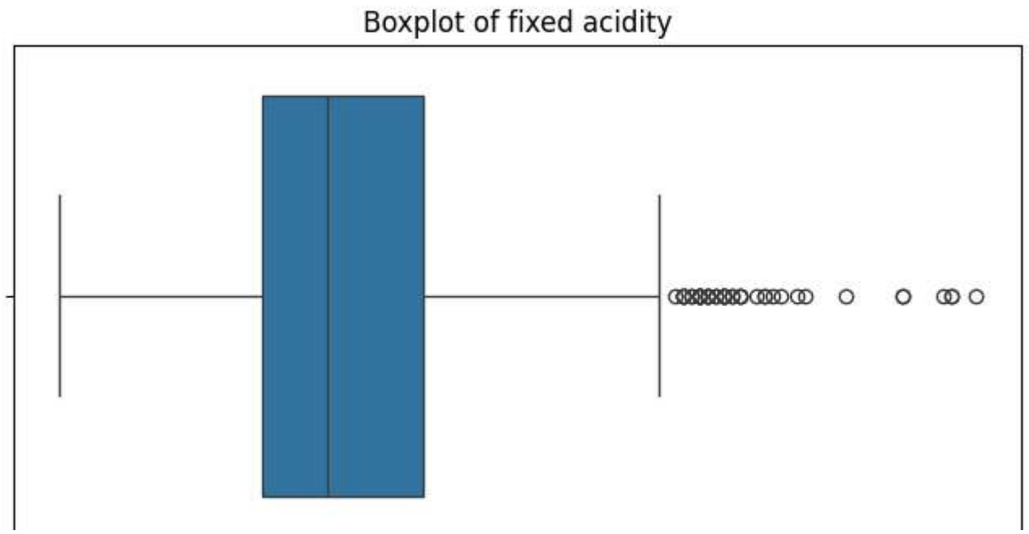
numeric_cols=df.columns

for col in numeric_cols:
    plt.figure(figsize=(7,4))
    sns.histplot(df[col],kde=True)
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()
```



In [12]:

```
# Boxplot-Outlier Ananlysis
for col in numeric_cols:
    plt.figure(figsize=(8,4))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.show()
```



In [13]:

```
# Detect outliers for every feature
features=df.drop(columns=["quality"])

#Calculate Q1 and Q3
Q1=features.quantile(0.25)
Q3=features.quantile(0.75)

# Calculate IQR
IQR=Q3-Q1

#Calculate Lower and Upper Limits
lower_bound=Q1-1.5*IQR
upper_bound=Q3+1.5*IQR

# Detect Outliers
outlier_mask=(features<lower_bound) | (features>upper_bound)

#Count outliers for each feature
outlier_count=outlier_mask.sum()

#Percentage of Outliers
outlier_percentage=(outlier_count/len(features))*100

#Create Summary table
outlier_summary=pd.DataFrame({
    'Q1':Q1,
    'Q3':Q3,
    'IQR':IQR,
    'lower bound':lower_bound,
    'upper bound':upper_bound,
    'Outlier Count':outlier_count,
    'Outlier %':outlier_percentage
})

outlier_summary.round(3)
```

Out[13]:

	Q1	Q3	IQR	lower bound	upper bound	Outlier Count	Outlier %
fixed acidity	7.100	9.100	2.000	4.100	12.100	44	3.850
volatile acidity	0.392	0.640	0.248	0.021	1.011	14	1.225
citric acid	0.090	0.420	0.330	-0.405	0.915	1	0.087
residual sugar	1.900	2.600	0.700	0.850	3.650	110	9.624
chlorides	0.070	0.090	0.020	0.040	0.120	77	6.737
free sulfur dioxide	7.000	21.000	14.000	-14.000	42.000	18	1.575
total sulfur dioxide	21.000	61.000	40.000	-39.000	121.000	40	3.500
density	0.996	0.998	0.002	0.992	1.001	36	3.150
pH	3.205	3.400	0.195	2.913	3.692	20	1.750
sulphates	0.550	0.730	0.180	0.280	1.000	43	3.762
alcohol	9.500	11.100	1.600	7.100	13.500	12	1.050
Id	411.000	1209.500	798.500	-786.750	2407.250	0	0.000

In [14]:

```
#No of outliers in each feature

print("Outliers in each feature:\n")

for column in features.columns:
    print(f"{column:25s}:{outlier_count[column]}")
```

```
Outliers in each feature:

fixed acidity            :44
volatile acidity         :14
citric acid              :1
residual sugar           :110
chlorides                :77
free sulfur dioxide      :18
total sulfur dioxide     :40
density                  :36
pH                       :20
sulphates                :43
alcohol                  :12
Id                       :0
```

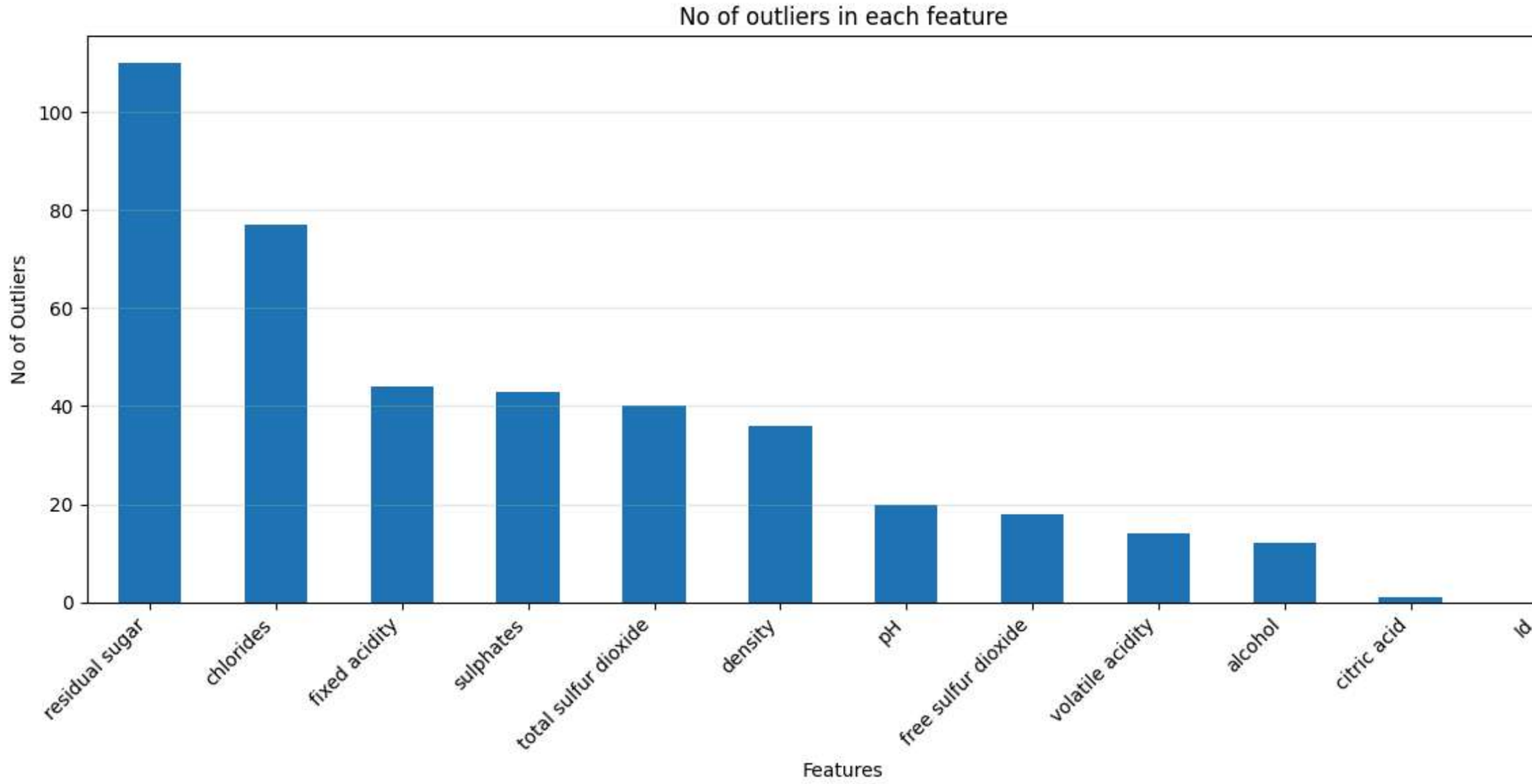
In [15]:

```
plt.figure(figsize=(12,6))

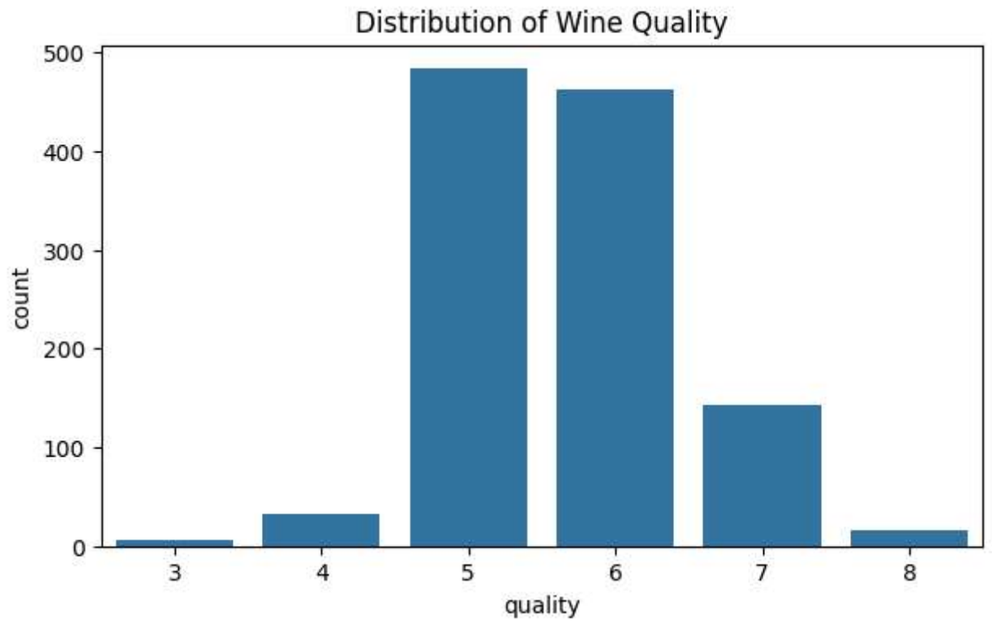
outlier_count.sort_values(ascending=False).plot(
    kind='bar'
)

plt.title("No of outliers in each feature")
plt.xlabel("Features")
plt.ylabel("No of Outliers")
plt.xticks(rotation=45,ha='right')
plt.grid(axis='y',alpha=0.3)

plt.tight_layout()
plt.show()
```


No of outliers in each feature
No of Outliers
0
20
40
60
80
100
residual sugar
chlorides
fixed acidity
sulphates
total sulfur dioxide
density
pH
free sulfur dioxide
volatile acidity
alcohol
citric acid
Features

In [16]:
```
#Quality Distribution
plt.figure(figsize=(7,4))
sns.countplot(x='quality',data=df)
plt.title("Distribution of Wine Quality")
plt.show()
```

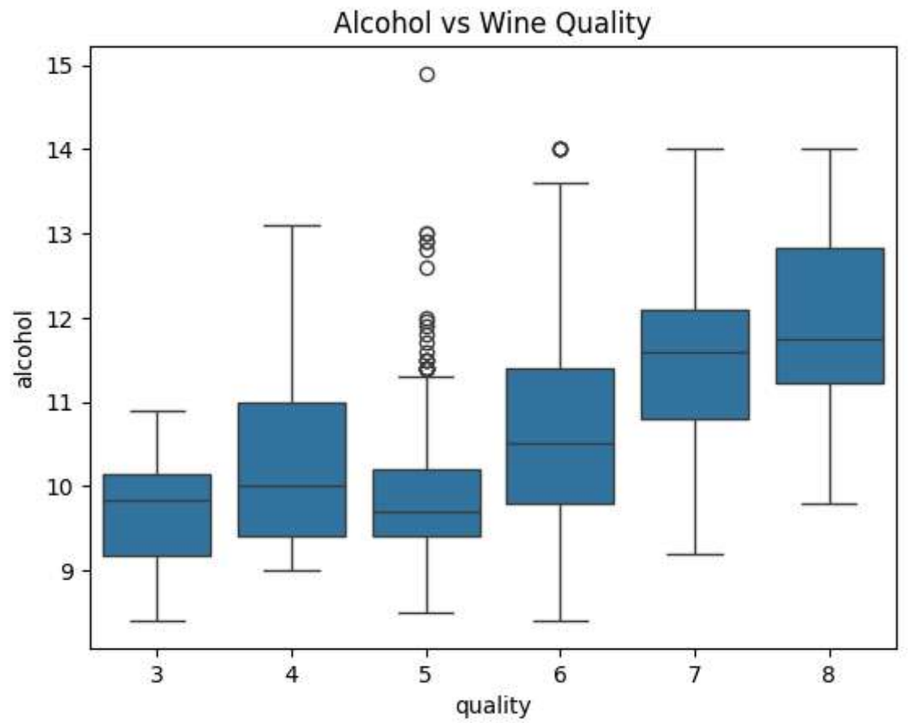


In [17]:
```
df['quality'].value_counts().sort_index()
```

Out[17]:
```
quality
3      6
4     33
5    483
6    462
7    143
8     16
Name: count, dtype: int64
```

2.Bivariate Analysis

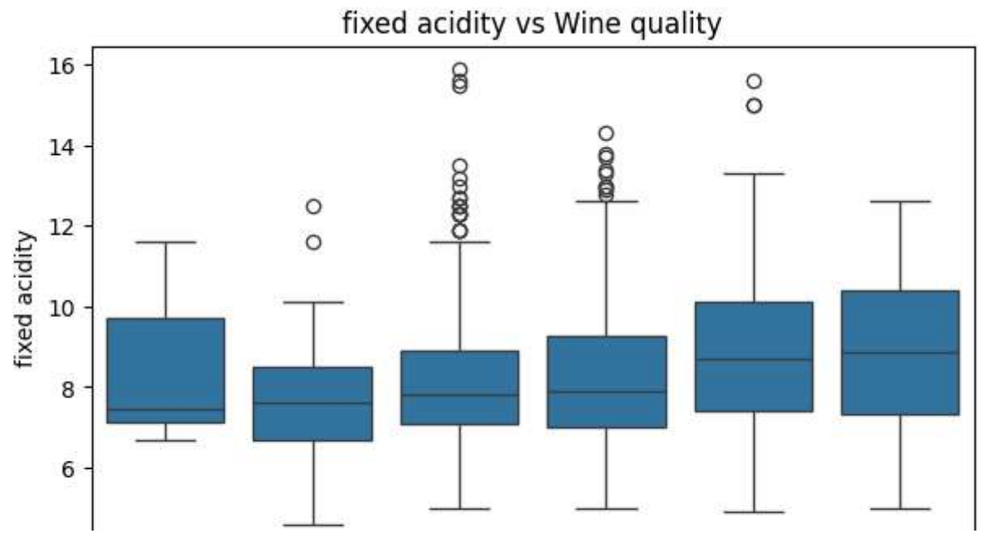
In [18]:
```
#Alcohol vs Quality
sns.boxplot(x='quality',y='alcohol',data=df)
plt.title('Alcohol vs Wine Quality')
plt.show()
```



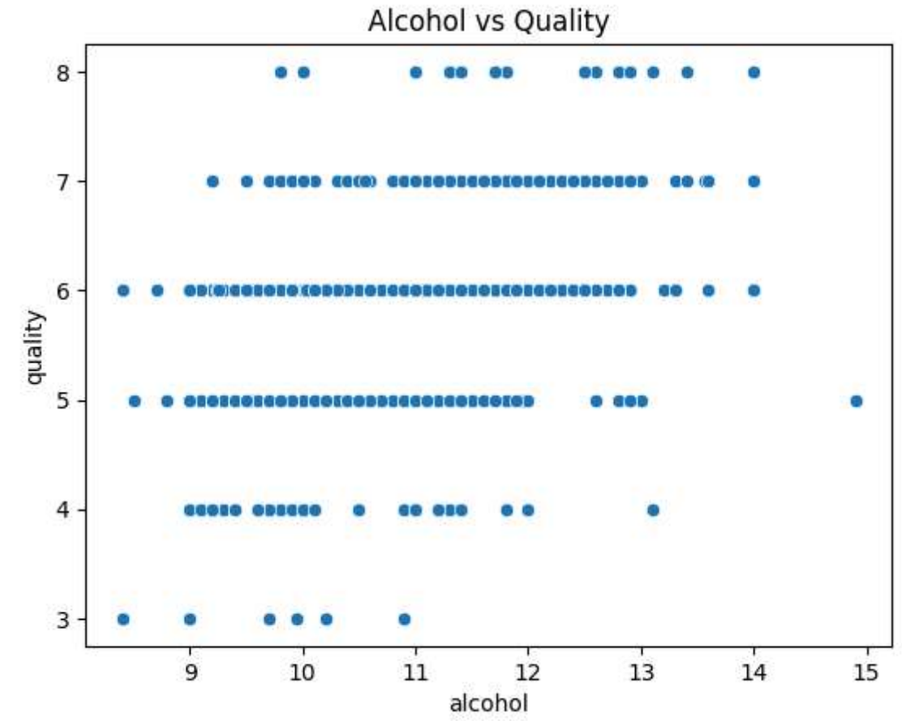
In [19]:

```
features=['fixed acidity','volatile acidity','citric acid','residual sugar','chlorides','free sulfur dioxide','total sulfur dioxide',
          'density','pH','sulphates','alcohol']

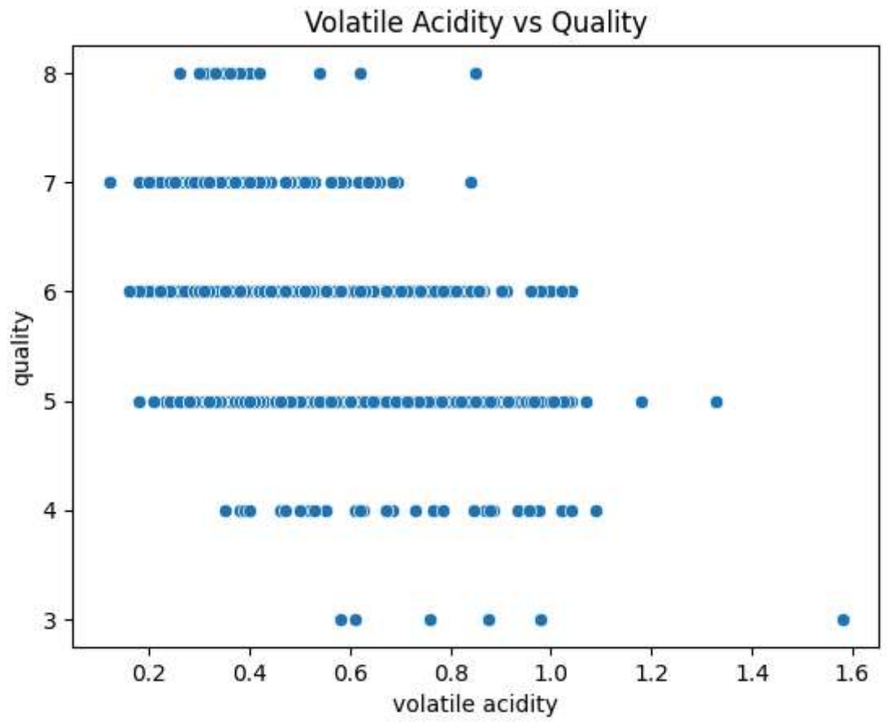
for col in features:
    plt.figure(figsize=(7,4))
    sns.boxplot(x='quality',y=col,data=df)
    plt.title(f'{col} vs Wine quality')
    plt.show()
```



In [20]:
```
#Scatter Plot
sns.scatterplot(x='alcohol',y='quality',data=df)
plt.title('Alcohol vs Quality')
plt.show()
```



In [21]:
```
sns.scatterplot(x='volatile acidity',y='quality',data=df)
plt.title('Volatile Acidity vs Quality')
plt.show()
```



3.Multivariate Analysis

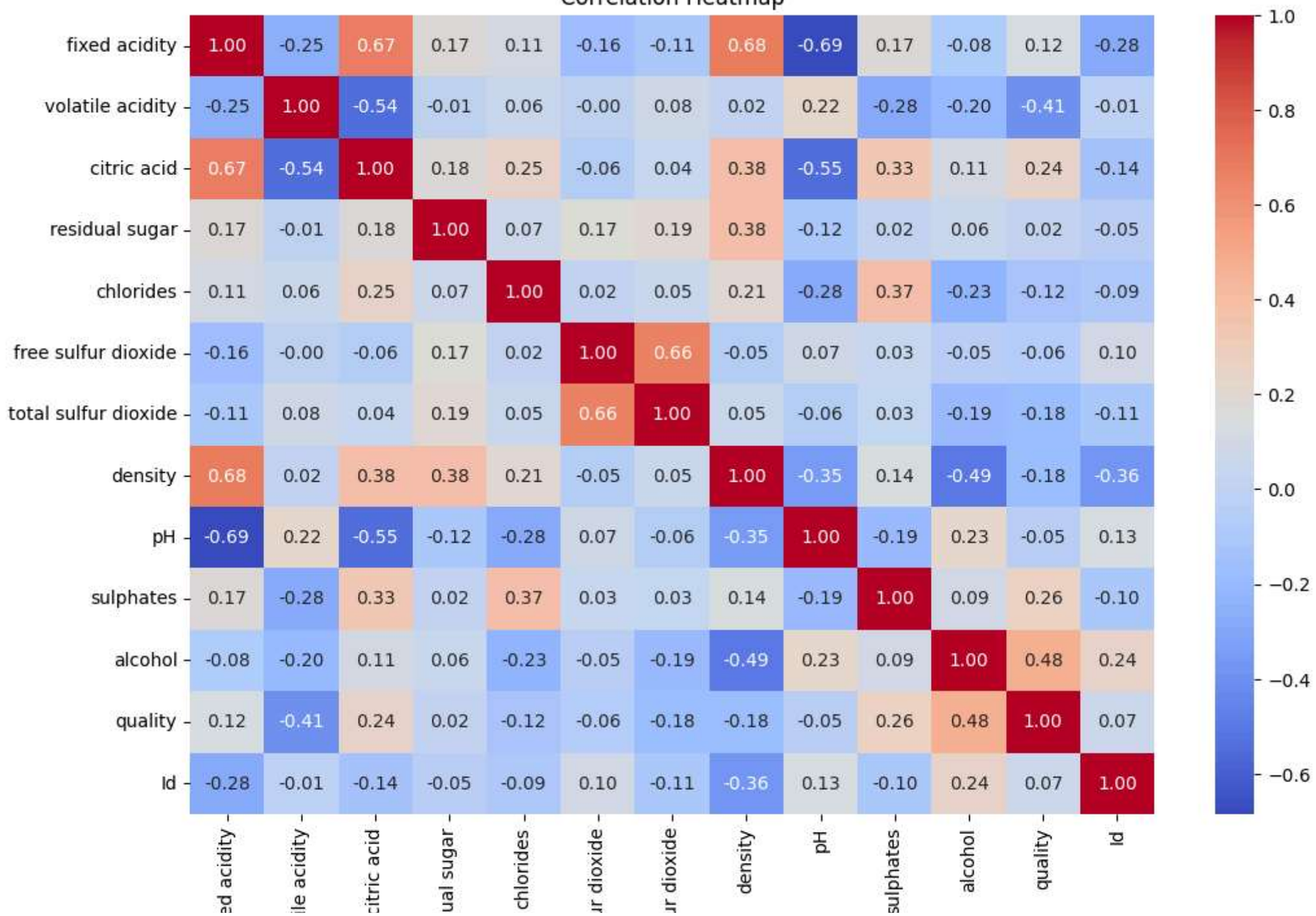
In [22]:

```
plt.figure(figsize=(12,8))

corr=df.corr()

sns.heatmap(corr,annot=True,cmap='coolwarm',fmt='.2f')

plt.title('Correlation Heatmap')
plt.show()
```


Correlation Heatmap
fixed acidity 1.00 -0.25 0.67 0.17 0.11 -0.16 -0.11 0.68 -0.69 0.17 -0.08 0.12 -0.28
volatile acidity -0.25 1.00 -0.54 -0.01 0.06 -0.00 0.08 0.02 0.22 -0.28 -0.20 -0.41 -0.01
citric acid 0.67 -0.54 1.00 0.18 0.25 -0.06 0.04 0.38 -0.55 0.33 0.11 0.24 -0.14
residual sugar 0.17 -0.01 0.18 1.00 0.07 0.17 0.19 0.38 -0.12 0.02 0.06 0.02 -0.05
chlorides 0.11 0.06 0.25 0.07 1.00 0.02 0.05 0.21 -0.28 0.37 -0.23 -0.12 -0.09
free sulfur dioxide -0.16 -0.00 -0.06 0.17 0.02 1.00 0.66 -0.05 0.07 0.03 -0.05 -0.06 0.10
total sulfur dioxide -0.11 0.08 0.04 0.19 0.05 0.66 1.00 0.05 -0.06 0.03 -0.19 -0.18 -0.11
density 0.68 0.02 0.38 0.38 0.21 -0.05 0.05 1.00 -0.35 0.14 -0.49 -0.18 -0.36
pH -0.69 0.22 -0.55 -0.12 -0.28 0.07 -0.06 -0.35 1.00 -0.19 0.23 -0.05 0.13
sulphates 0.17 -0.28 0.33 0.02 0.37 0.03 0.03 0.14 -0.19 1.00 0.09 0.26 -0.10
alcohol -0.08 -0.20 0.11 0.06 -0.23 -0.05 -0.19 -0.49 0.23 0.09 1.00 0.48 0.24
quality 0.12 -0.41 0.24 0.02 -0.12 -0.06 -0.18 -0.18 -0.05 0.26 0.48 1.00 0.07
Id -0.28 -0.01 -0.14 -0.05 -0.09 0.10 -0.11 -0.36 0.13 -0.10 0.24 0.07 1.00
ed acidity
ile acidity
citric acid
ual sugar
chlorides
r dioxide
r dioxide
density
pH
sulphates
alcohol
quality
Id
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

fixe

volati

c

resid

free sulfu

total sulfu

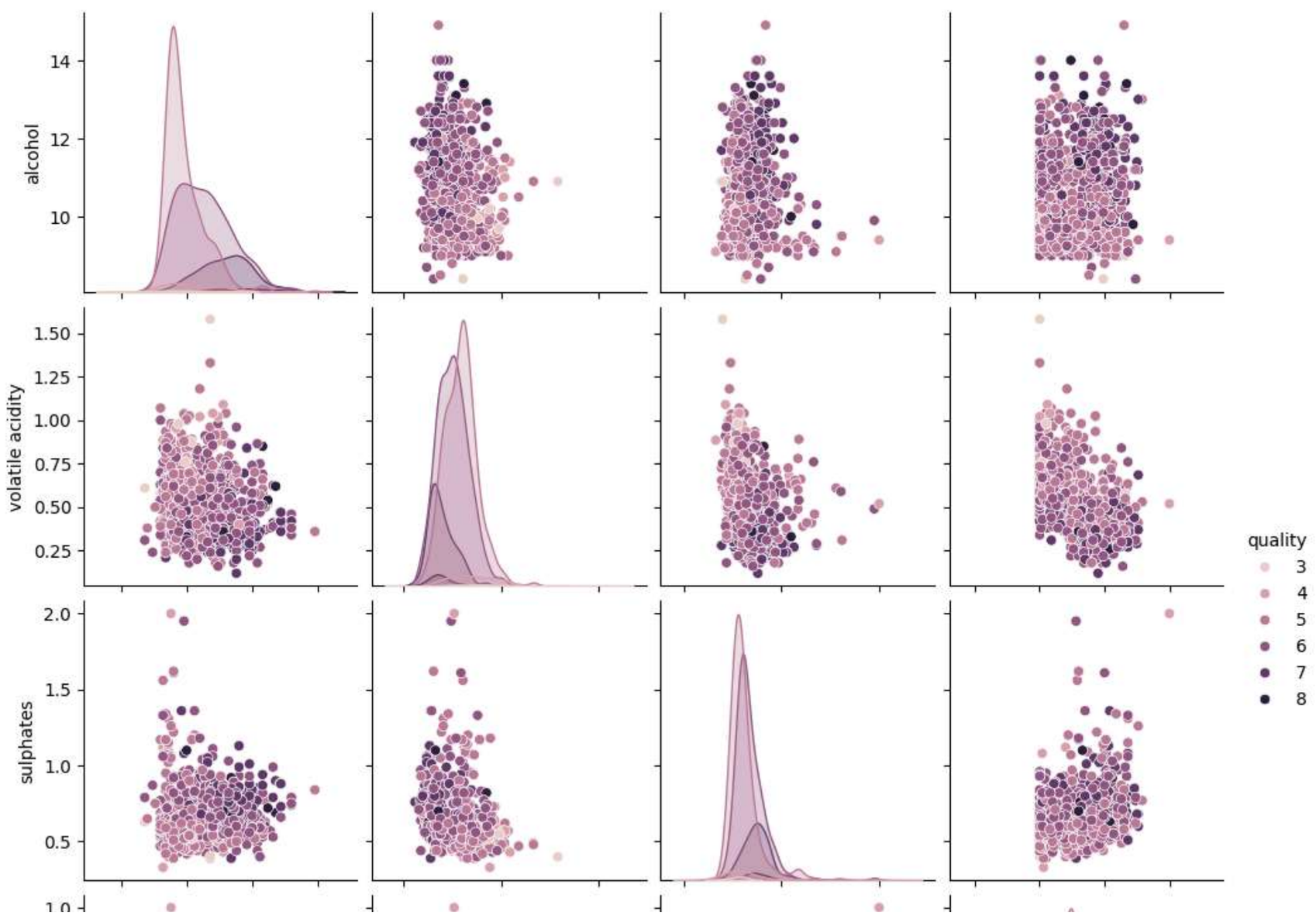
s

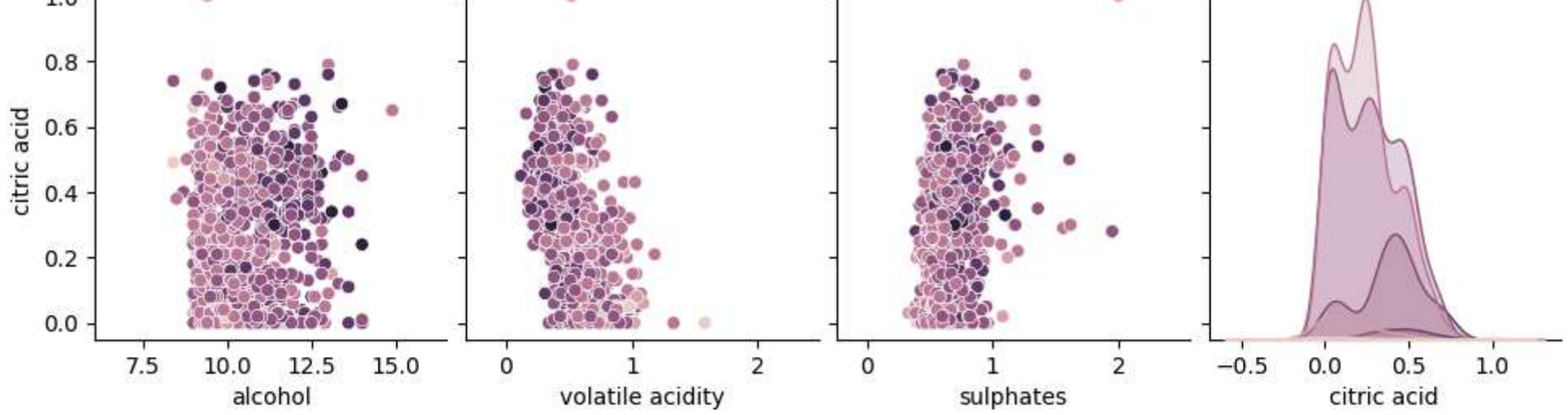
In [23]:

```
# Pair Plot
selected=['alcohol','volatile acidity','sulphates','citric acid','quality']

sns.pairplot(df[selected],hue='quality')

plt.show()
```


alcohol
volatile acidity
sulphates
14
12
10
1.50
1.25
1.00
0.75
0.50
0.25
2.0
1.5
1.0
0.5
quality
3
4
5
6
7
8



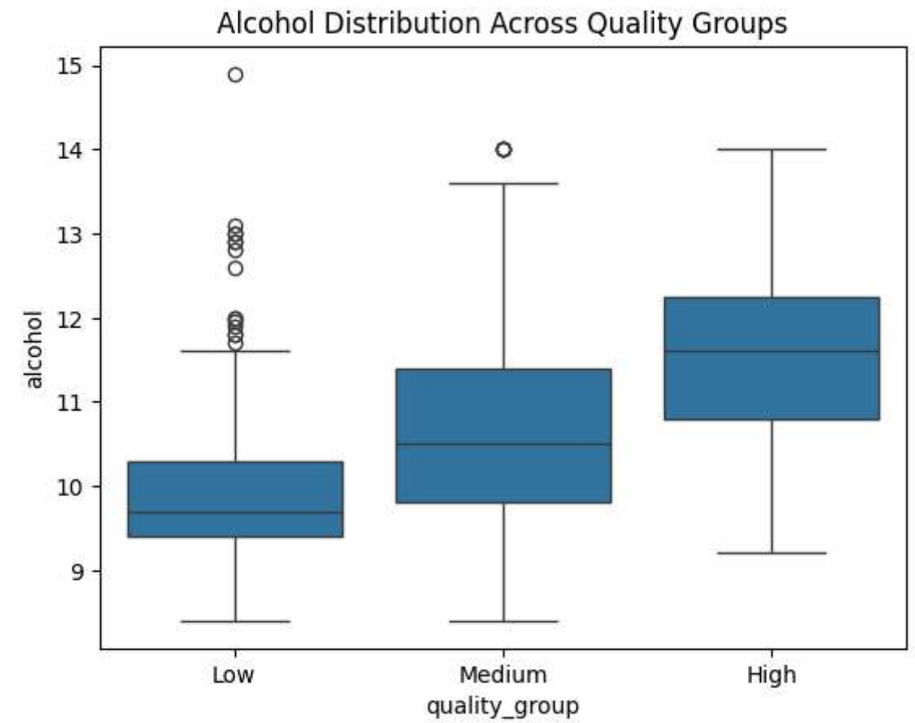
In [24]:

```
#Multivariate Ananlysis Using Quality Groups
df['quality_group']=pd.cut(
    df['quality'],
    bins=[0,5,6,10],
    labels=['Low','Medium','High']
)
```

In [25]:

```
sns.boxplot(
    x='quality_group',
    y='alcohol',
    data=df
)

plt.title('Alcohol Distribution Across Quality Groups')
plt.show()
```



In [26]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier,plot_tree
from sklearn.metrics import (accuracy_score,precision_score,recall_score,
                             f1_score,classification_report,confusion_matrix,ConfusionMatrixDisplay)
```

In [27]:
```
X=df.drop('quality',axis=1)
y=df['quality']
```

In [28]:
```
print(y.value_counts())
```

```
quality
5    483
6    462
7    143
4     33
8     16
3      6
Name: count, dtype: int64
```

In [29]:
```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.20,
                                               random_state=42,
                                               stratify=y
                                              )
```

In [30]:
```
criterion='entropy'
```

In [31]:
```
dt_model=DecisionTreeClassifier(
    criterion='entropy',
    random_state=42)
```

In [32]:
```
df['quality_group']=pd.cut(
    df['quality'],
    bins=[0,5,6,10],
    labels=['Low','Medium','High']
)
```

In [33]:
```
print(X.columns)
```

```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol', 'Id', 'quality_group'],
      dtype='object')
```

In [34]:
```
print(X.select_dtypes(exclude='number').columns)
```

```
Index(['quality_group'], dtype='object')
```

In [35]:
```
y=df['quality']
```

In [36]:
```
X=df.drop(columns=['quality','quality_group'],errors='ignore')
```

In [37]:
```
print(X.columns)
print(X.dtypes)
```

```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol', 'Id'],
      dtype='object')
fixed acidity           float64
volatile acidity        float64
citric acid             float64
residual sugar          float64
chlorides               float64
free sulfur dioxide     float64
total sulfur dioxide    float64
density                 float64
pH                      float64
sulphates               float64
alcohol                 float64
Id                        int64
dtype: object
```

In [38]:
```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.20,
                                               random_state=42,
                                               stratify=y
                                              )
```

In [ ]:
```
```

In [39]:
```
dt_model=DecisionTreeClassifier(criterion='entropy',random_state=42)
dt_model.fit(X_train,y_train)
```

Out[39]:
```
DecisionTreeClassifier(criterion='entropy', random_state=42)
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [40]:
```
dt_model.fit(X_train,y_train)
```

Out[40]:
```
DecisionTreeClassifier(criterion='entropy', random_state=42)
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [41]:
```
y_train_pred=dt_model.predict(X_train)
y_test_pred=dt_model.predict(X_test)
```

In [42]:
```
train_accuracy=accuracy_score(y_train,y_train_pred)

print("Training Accuracy:",train_accuracy)
```

```
Training Accuracy: 1.0
```

In [43]:
```
test_accuracy=accuracy_score(y_test,y_test_pred)

print("Testing Accuracy:",test_accuracy)
```

```
Testing Accuracy: 0.6200873362445415
```

In [44]:
```
print("Precision:",
      precision_score(y_test,y_test_pred,average='weighted'))
print("Recall:",
      recall_score(y_test,y_test_pred,average='weighted'))
print("F1 Score:",
      f1_score(y_test,y_test_pred,average='weighted'))
```

```
Precision: 0.6070017244204808
Recall: 0.6200873362445415
F1 Score: 0.6099421427959831

C:\Users\prabh\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-de
nd being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In [45]:
```
print(classification_report(y_test,y_test_pred))
```

```
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.20      0.14      0.17         7
           5       0.69      0.80      0.74        97
           6       0.62      0.52      0.56        92
           7       0.48      0.52      0.50        29
           8       0.00      0.00      0.00         3

    accuracy                           0.62       229
   macro avg       0.33      0.33      0.33       229
weighted avg       0.61      0.62      0.61       229


C:\Users\prabh\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-de
nd being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\prabh\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-de
nd being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\prabh\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classification.py:1879: UndefinedMetricWarning: Precision is ill-de
nd being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In [46]:

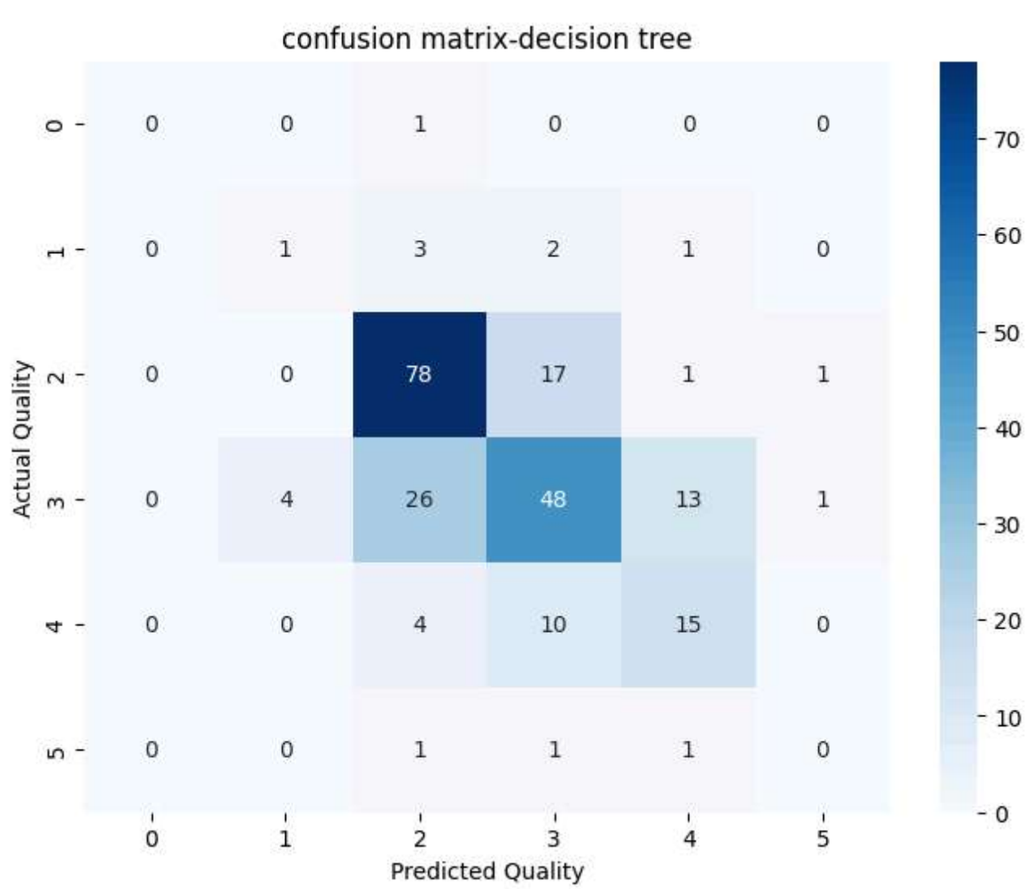
```
cm=confusion_matrix(y_test,y_test_pred)

plt.figure(figsize=(8,6))

sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')

plt.xlabel('Predicted Quality')
plt.ylabel('Actual Quality')
plt.title('confusion matrix-decision tree')

plt.show()
```



In [47]:
```
# Decision Trees
```

In [48]:

```
print('tree depth', dt_model.get_depth())
print('number of leaves: ',dt_model.get_n_leaves())
```

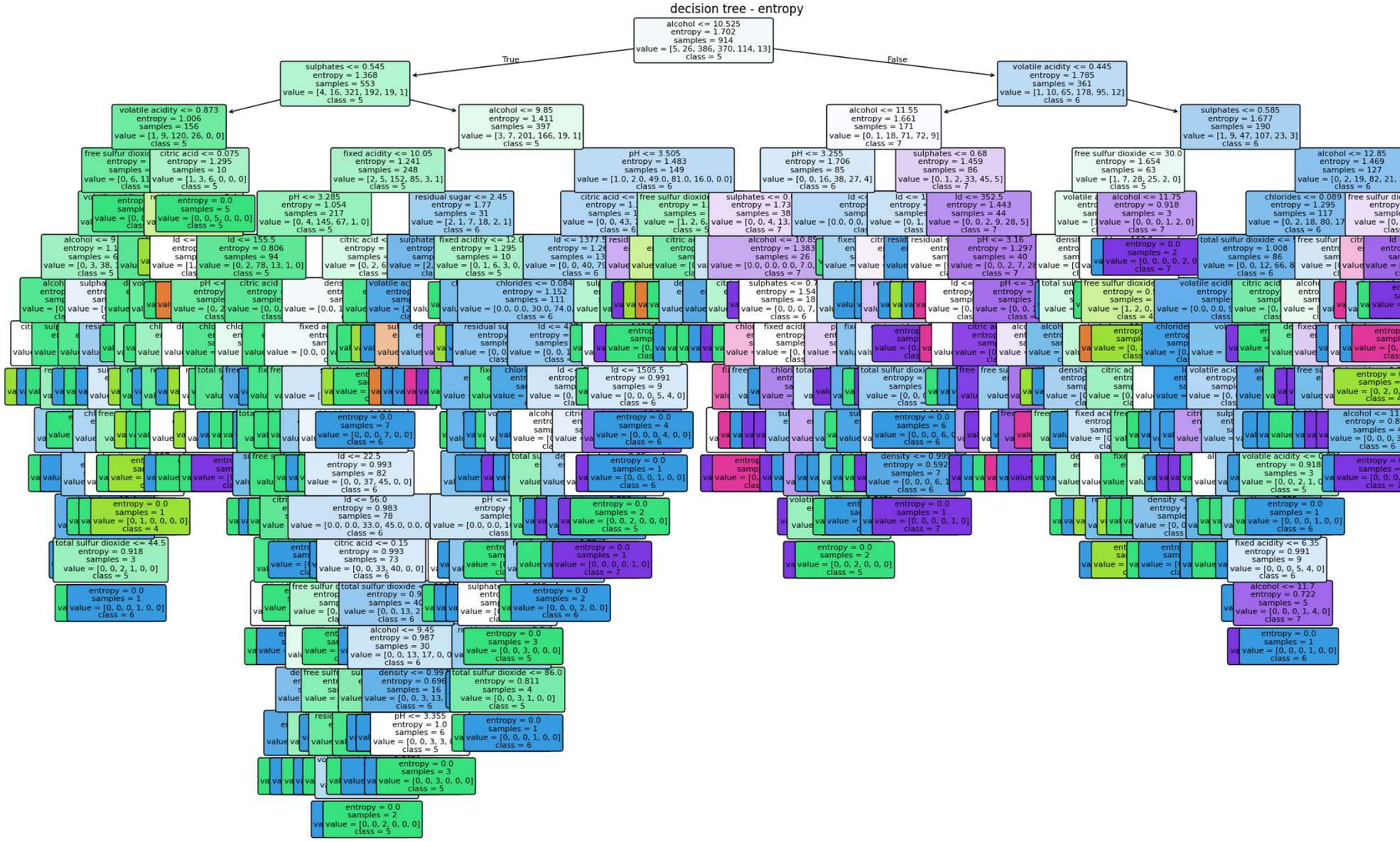
```
tree depth 18
number of leaves:  236
```

In [49]:

```
plt.figure(figsize=(25,15))

plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=[str(c) for c in sorted(y.unique())],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title('decision tree - entropy')
plt.show()
```


decision tree - entropy
alcohol <= 10.525
entropy = 1.702
samples = 914
value = [5, 26, 386, 370, 114, 13]
class = 5
True
False
sulphates <= 0.545
entropy = 1.368
samples = 553
value = [4, 16, 321, 192, 19, 1]
class = 5
volatile acidity <= 0.445
entropy = 1.785
samples = 361
value = [1, 10, 65, 178, 95, 12]
class = 6
volatile acidity <= 0.873
entropy = 1.006
samples = 156
value = [1, 9, 120, 26, 0, 0]
class = 5
alcohol <= 9.85
entropy = 1.411
samples = 397
value = [3, 7, 201, 166, 19, 1]
class = 5
alcohol <= 11.55
entropy = 1.661
samples = 171
value = [0, 1, 18, 71, 72, 9]
class = 7
sulphates <= 0.585
entropy = 1.677
samples = 190
value = [1, 9, 47, 107, 23, 3]
class = 6
free sulfur dioxide
entropy =
samples = 10
value = [0, 6, 1
class =
citric acid <= 0.075
entropy = 1.295
samples = 10
value = [1, 3, 6, 0, 0, 0]
class = 5
entropy = 0.0
samples = 5
value = [0, 0, 5, 0, 0, 0]
class = 5
fixed acidity <= 10.05
entropy = 1.241
samples = 248
value = [2, 5, 152, 85, 3, 1]
class = 5
pH <= 3.285
entropy = 1.054
samples = 217
value = [0, 4, 145, 67, 1, 0]
class = 5
residual sugar <= 2.45
entropy = 1.77
samples = 31
value = [2, 1, 7, 18, 2, 1]
class = 6
pH <= 3.505
entropy = 1.483
samples = 149
value = [1.0, 2.0, 49.0, 81.0, 16.0, 0.0]
class = 6
pH <= 3.255
entropy = 1.706
samples = 85
value = [0, 0, 16, 38, 27, 4]
class = 6
sulphates <= 0.68
entropy = 1.459
samples = 86
value = [0, 1, 2, 33, 45, 5]
class = 7
free sulfur dioxide <= 30.0
entropy = 1.654
samples = 63
value = [1, 7, 28, 25, 2, 0]
class = 5
alcohol <= 12.85
entropy = 1.469
samples = 127
value = [0, 2, 19, 82, 21,
total sulfur dioxide <= 44.5
entropy = 0.918
samples = 3
value = [0, 0, 2, 1, 0, 0]
class = 5
entropy = 0.0
samples = 1
value = [0, 0, 0, 1, 0, 0]
class = 6
entropy = 0.0
samples = 1
value = [0, 1, 0, 0, 0, 0]
class = 4
entropy = 0.0
samples = 7
value = [0, 0, 0, 7, 0, 0]
class = 6
entropy = 0.0
samples = 4
value = [0, 0, 0, 4, 0, 0]
class = 6
entropy = 0.0
samples = 6
value = [0, 0, 0, 6, 0,
class = 6
entropy = 0.0
samples = 1
value = [0, 0, 0, 0, 1, 0]
class = 6
entropy = 0.0
samples = 1
value = [0, 0, 0, 0, 1, 0]
class = 7
entropy = 0.0
samples = 2
value = [0, 0, 2, 0, 0, 0]
class = 5
entropy = 0.0
samples = 2
value = [0, 0, 2, 0, 0, 0]
class = 5
entropy = 0.0
samples = 1
value = [0, 0, 0, 1, 0, 0]
class = 6
entropy = 0.0
samples = 1
value = [0, 0, 0, 1, 0, 0]
class = 6
fixed acidity <= 6.35
entropy = 0.991
samples = 9
value = [0, 0, 0, 5, 4, 0]
class = 6
alcohol <= 11.7
entropy = 0.722
samples = 5
value = [0, 0, 0, 1, 4, 0]
class = 7
entropy = 0.0
samples = 1
value = [0, 0, 0, 1, 0, 0]
class = 6

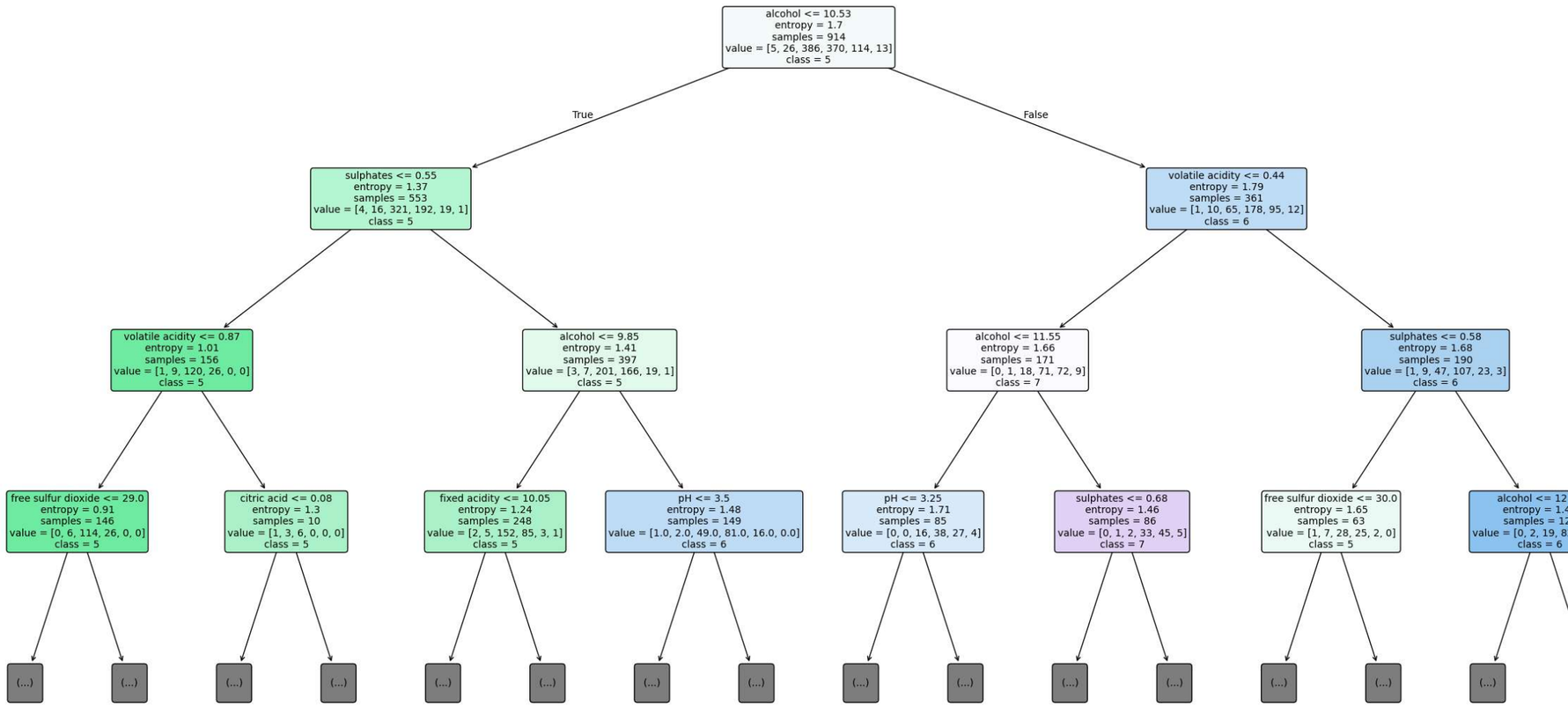
In [50]:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(24, 12))

plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=[str(c) for c in sorted(y.unique())],
    filled=True,
    rounded=True,
    impurity=True,
    precision=2,
    fontsize=10,
    max_depth=3
)

plt.title('Decision Tree - Top 3 Levels', fontsize=18)
plt.tight_layout()
plt.show()
```


Decision Tree - Top 3 Levels
alcohol <= 10.53
entropy = 1.7
samples = 914
value = [5, 26, 386, 370, 114, 13]
class = 5
True
False
sulphates <= 0.55
entropy = 1.37
samples = 553
value = [4, 16, 321, 192, 19, 1]
class = 5
volatile acidity <= 0.44
entropy = 1.79
samples = 361
value = [1, 10, 65, 178, 95, 12]
class = 6
volatile acidity <= 0.87
entropy = 1.01
samples = 156
value = [1, 9, 120, 26, 0, 0]
class = 5
alcohol <= 9.85
entropy = 1.41
samples = 397
value = [3, 7, 201, 166, 19, 1]
class = 5
alcohol <= 11.55
entropy = 1.66
samples = 171
value = [0, 1, 18, 71, 72, 9]
class = 7
sulphates <= 0.58
entropy = 1.68
samples = 190
value = [1, 9, 47, 107, 23, 3]
class = 6
free sulfur dioxide <= 29.0
entropy = 0.91
samples = 146
value = [0, 6, 114, 26, 0, 0]
class = 5
citric acid <= 0.08
entropy = 1.3
samples = 10
value = [1, 3, 6, 0, 0, 0]
class = 5
fixed acidity <= 10.05
entropy = 1.24
samples = 248
value = [2, 5, 152, 85, 3, 1]
class = 5
pH <= 3.5
entropy = 1.48
samples = 149
value = [1.0, 2.0, 49.0, 81.0, 16.0, 0.0]
class = 6
pH <= 3.25
entropy = 1.71
samples = 85
value = [0, 0, 16, 38, 27, 4]
class = 6
sulphates <= 0.68
entropy = 1.46
samples = 86
value = [0, 1, 2, 33, 45, 5]
class = 7
free sulfur dioxide <= 30.0
entropy = 1.65
samples = 63
value = [1, 7, 28, 25, 2, 0]
class = 5
(...)

auc and roc

In [51]:

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc, roc_auc_score
import matplotlib.pyplot as plt
import numpy as np

# get unique classes
classes = sorted(y_test.unique())

# convert y_test to binary format
y_test_bin = label_binarize(y_test, classes=classes)

# get probability predictions
y_score = dt_model.predict_proba(X_test)

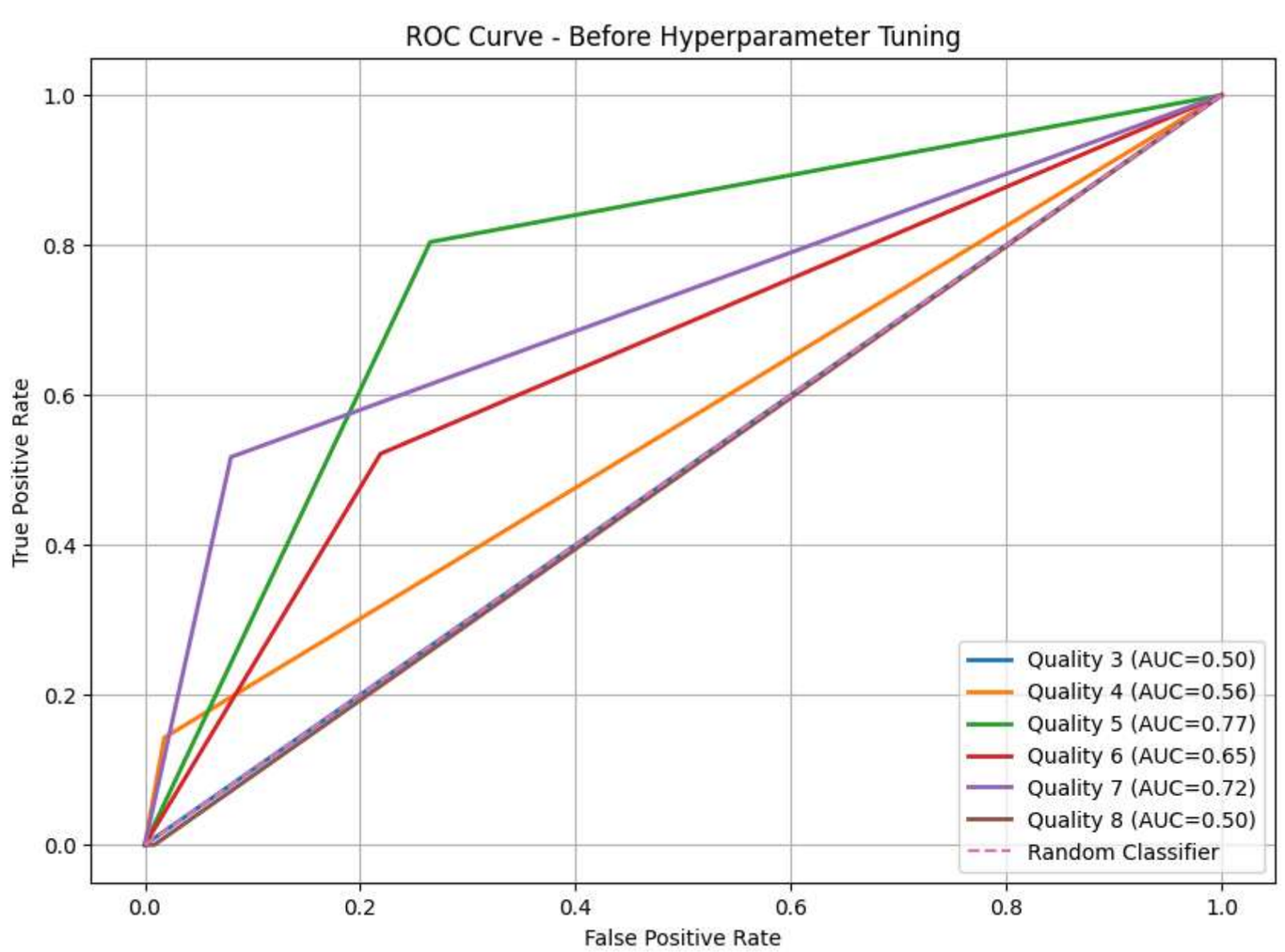
# plot roc curve
plt.figure(figsize=(10, 7))

for i, class_label in enumerate(classes):
    fpr, tpr, thresholds = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, linewidth=2,
             label=f'Quality {class_label} (AUC={roc_auc:.2f})')

# random classifier line
plt.plot([0, 1], [0, 1], linestyle='--', label='Random Classifier')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Before Hyperparameter Tuning')
plt.legend()
plt.grid()
plt.show()
```

```
# overall AUC
overall_auc_before = roc_auc_score(y_test_bin, y_score, multi_class='ovr', average='weighted')
print('Overall ROC AUC before tuning:', round(overall_auc_before, 4))
```




```
Overall ROC AUC before tuning: 0.7045
```

hyper parameter tuning

In [52]:

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
import warnings
warnings.filterwarnings('ignore')

param_grid={
    'criterion': ['entropy'],
    'max_depth': [3,5,7,10,None],
    'min_samples_split': [2,5,10,20],
    'min_samples_leaf': [1,2,4,8],
    'max_features': [None,'sqrt','log2'],
    'ccp_alpha': [0,0.001,0.005,0.01]
}

grid= GridSearchCV(DecisionTreeClassifier(random_state=42),param_grid,cv=5,scoring='accuracy',n_jobs=-1)

grid.fit(X_train,y_train)

print('best parameters: ')
print(grid.best_params_)

print("best cross validation accuracy")
print(grid.best_score_)

best_tree=grid.best_estimator_
```

```
best parameters:
{'ccp_alpha': 0, 'criterion': 'entropy', 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 8, 'min_samples_split': 20}
best cross validation accuracy
0.5787785984507295
```

In [54]:
```
y_pred=best_tree.predict(X_test)

print(classification_report(y_test,y_pred))
print("Accuracy = ", accuracy_score(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00         7
           5       0.72      0.73      0.72        97
           6       0.56      0.66      0.61        92
           7       0.35      0.24      0.29        29
           8       0.00      0.00      0.00         3

    accuracy                           0.61       229
   macro avg       0.27      0.27      0.27       229
weighted avg       0.57      0.61      0.59       229

Accuracy =  0.6069868995633187
```

In [56]:
```
#training accuracy
training_accuracy=best_tree.score(X_train,y_train)

#testing accuracy
testing_accuracy=best_tree.score(X_test,y_test)

print('training accuracy : ',training_accuracy)
print('testing accuracy',testing_accuracy)
```

```
training accuracy :  0.6903719912472648
testing accuracy 0.6069868995633187
```

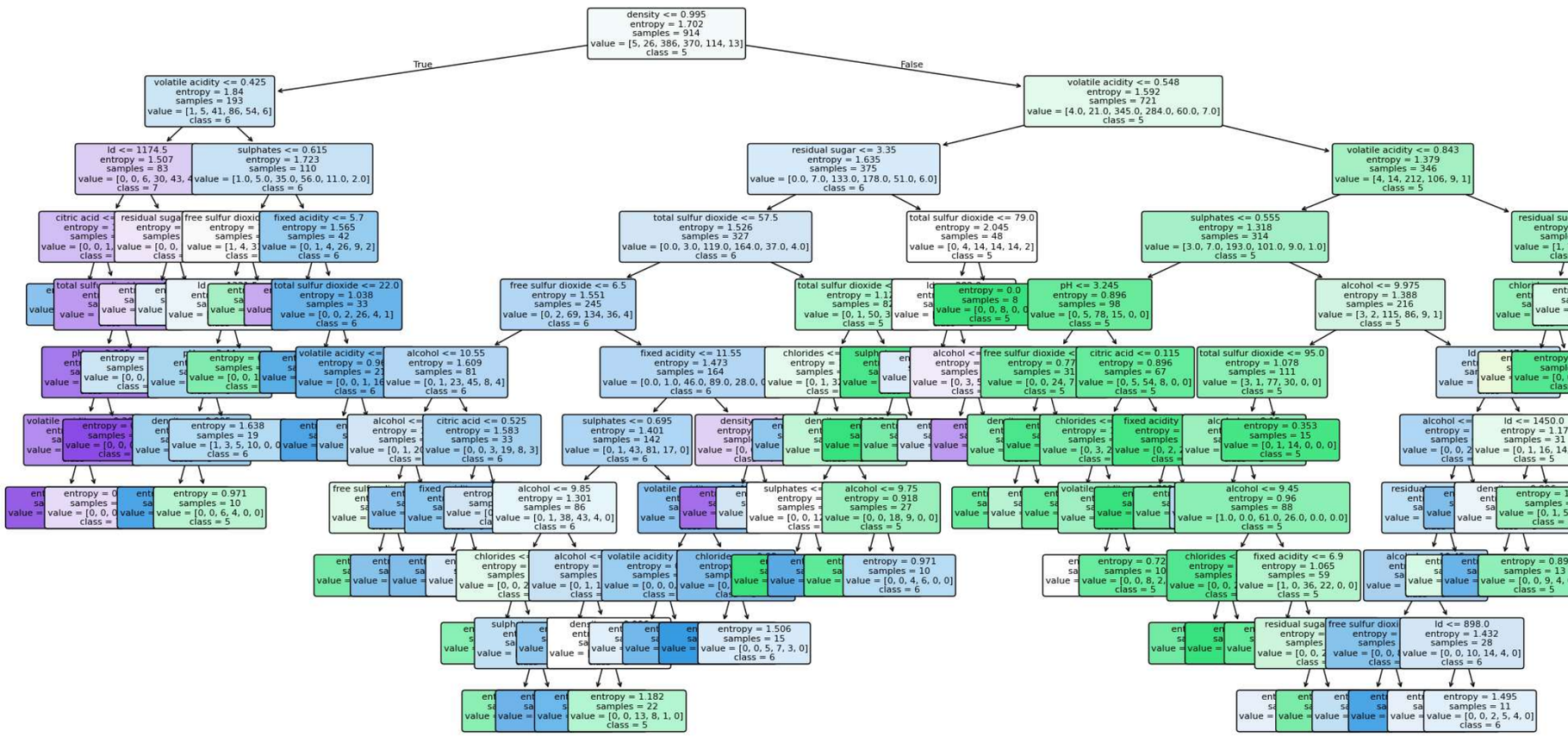
In [64]:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(25,12))

plot_tree(
    best_tree,
    feature_names=X.columns,
    class_names=[str (c) for c in best_tree.classes_],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title('decision tree after hyperparameter tuning')
plt.show()
```


decision tree after hyperparameter tuning
density <= 0.995
entropy = 1.702
samples = 914
value = [5, 26, 386, 370, 114, 13]
class = 5
True
False
volatile acidity <= 0.425
entropy = 1.84
samples = 193
value = [1, 5, 41, 86, 54, 6]
class = 6
volatile acidity <= 0.548
entropy = 1.592
samples = 721
value = [4.0, 21.0, 345.0, 284.0, 60.0, 7.0]
class = 5
ld <= 1174.5
entropy = 1.507
samples = 83
sulphates <= 0.615
entropy = 1.723
samples = 110
residual sugar <= 3.35
entropy = 1.635
samples = 375
value = [0.0, 7.0, 133.0, 178.0, 51.0, 6.0]
class = 6
volatile acidity <= 0.843
entropy = 1.379
samples = 346
value = [4, 14, 212, 106, 9, 1]
class = 5
total sulfur dioxide <= 57.5
entropy = 1.526
samples = 327
value = [0.0, 3.0, 119.0, 164.0, 37.0, 4.0]
class = 6
total sulfur dioxide <= 79.0
entropy = 2.045
samples = 48
value = [0, 4, 14, 14, 14, 2]
class = 5
sulphates <= 0.555
entropy = 1.318
samples = 314
value = [3.0, 7.0, 193.0, 101.0, 9.0, 1.0]
class = 5
free sulfur dioxide <= 6.5
entropy = 1.551
samples = 245
value = [0, 2, 69, 134, 36, 4]
class = 6
pH <= 3.245
entropy = 0.896
samples = 98
value = [0, 5, 78, 15, 0, 0]
class = 5
alcohol <= 9.975
entropy = 1.388
samples = 216
value = [3, 2, 115, 86, 9, 1]
class = 5
fixed acidity <= 11.55
entropy = 1.473
samples = 164
value = [0.0, 1.0, 46.0, 89.0, 28.0, 0
class = 6
free sulfur dioxide <
entropy = 0.77
samples = 31
value = [0, 0, 24, 7
class = 5
citric acid <= 0.115
entropy = 0.896
samples = 67
value = [0, 5, 54, 8, 0, 0]
class = 5
total sulfur dioxide <= 95.0
entropy = 1.078
samples = 111
value = [3, 1, 77, 30, 0, 0]
class = 5
sulphates <= 0.695
entropy = 1.401
samples = 142
value = [0, 1, 43, 81, 17, 0]
class = 6
entropy = 0.353
samples = 15
value = [0, 1, 14, 0, 0, 0]
class = 5
alcohol <= 9.85
entropy = 1.301
samples = 86
value = [0, 1, 38, 43, 4, 0]
class = 6
alcohol <= 9.75
entropy = 0.918
samples = 27
value = [0, 0, 18, 9, 0, 0]
class = 5
alcohol <= 9.45
entropy = 0.96
samples = 88
value = [1.0, 0.0, 61.0, 26.0, 0.0, 0.0]
class = 5
entropy = 1.506
samples = 15
value = [0, 0, 5, 7, 3, 0]
class = 6
entropy = 1.182
samples = 22
value = [0, 0, 13, 8, 1, 0]
class = 5
entropy = 0.971
samples = 10
value = [0, 0, 4, 6, 0, 0]
class = 6
entropy = 0.72
samples = 10
value = [0, 0, 8, 2,
class = 5
fixed acidity <= 6.9
entropy = 1.065
samples = 59
value = [1, 0, 36, 22, 0, 0]
class = 5
entropy = 1.495
samples = 11
value = [0, 0, 2, 5, 4, 0]
class = 6

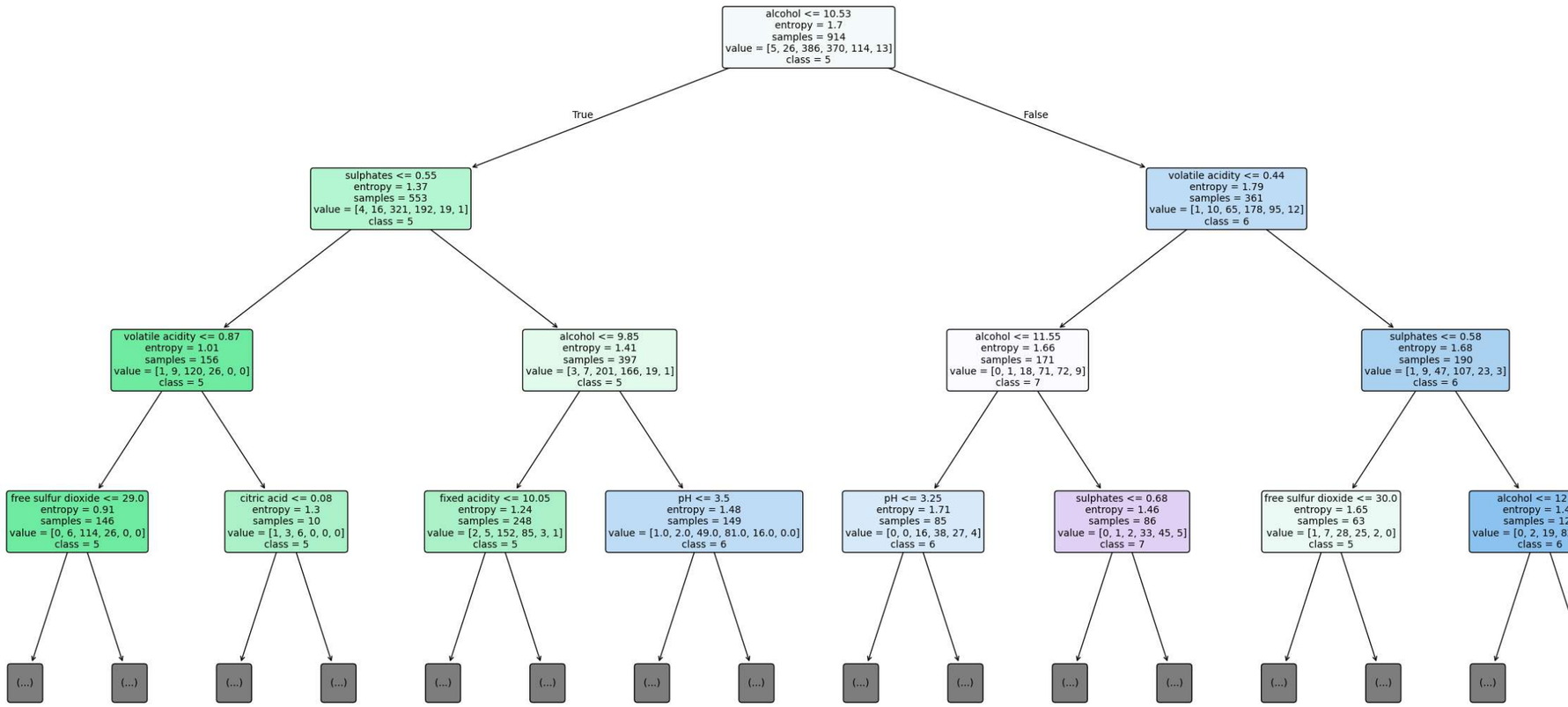
In [67]:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(24, 12))

plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=[str(c) for c in sorted(y.unique())],
    filled=True,
    rounded=True,
    impurity=True,
    precision=2,
    fontsize=10,
    max_depth=3
)

plt.title('Decision Tree - Top 3 Levels', fontsize=18)
plt.tight_layout()
plt.show()
```



cross validation

In [71]:

```
from sklearn.model_selection import cross_val_score

scores=cross_val_score(best_tree,X,y,cv=10,scoring='accuracy')

print(scores)
print('average accuracy=',scores.mean())
print("std = ", scores.std())
```

```
[0.46086957 0.60869565 0.53913043 0.44736842 0.53508772 0.59649123
 0.44736842 0.56140351 0.44736842 0.46491228]
average accuracy= 0.5108695652173914
std =  0.0612773576098809
```

learning curve

In [76]:

```
from sklearn.model_selection import learning_curve
import matplotlib.pyplot as plt

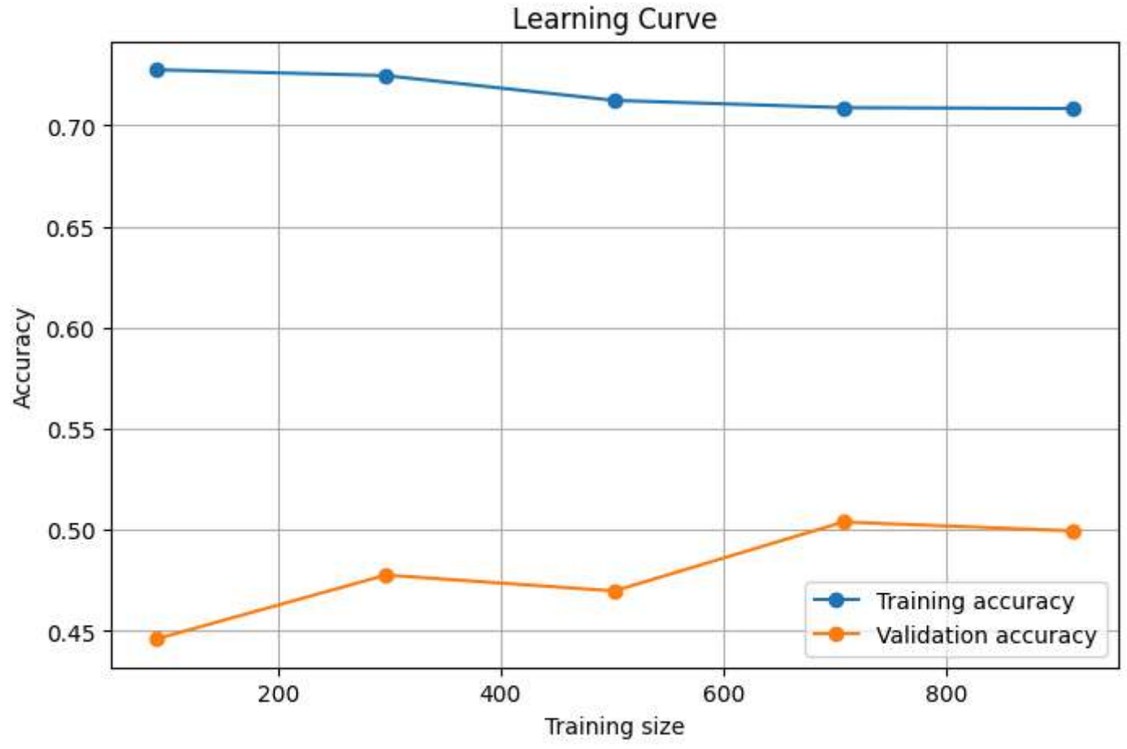
train_sizes, train_scores, test_scores = learning_curve(
    best_tree,
    X,
    y,
    cv=5,
    scoring='accuracy'
)

plt.figure(figsize=(8, 5))

plt.plot(train_sizes, train_scores.mean(axis=1),marker='o', label='Training accuracy')

plt.plot(train_sizes, test_scores.mean(axis=1),marker='o', label='Validation accuracy')

plt.xlabel("Training size")
plt.ylabel("Accuracy")
plt.title("Learning Curve")
plt.legend()
plt.grid()
plt.show()
```


Learning Curve
Accuracy
0.70
0.65
0.60
0.55
0.50
0.45
200
400
600
800
Training size
Training accuracy
Validation accuracy

In [87]:

```
new_wine = pd.DataFrame([{
            "fixed acidity": 1.0,
            "volatile acidity": 0.8,
            "citric acid": 0.4,
            "residual sugar": 2.0,
            "chlorides": 0.8,
            "free sulfur dioxide": 12.0,
            "total sulfur dioxide": 42.0,
            "density": 0.1,
            "pH": 1,
            "sulphates": 0.7,
            "alcohol": 9.0,
            "Id": 0
}])

predicted_quality = best_tree.predict(new_wine)
print("Predicted Quality:", predicted_quality[0])
```

```
Predicted Quality: 7
```

In [ ]: